

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 9
RELASI ANTAR OBJEK, ABSTRACT CLASS DAN
INTERFACE



Disusun Oleh:

Glory Hardy Febriando Sianturi

(105223013)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program sistem perbankan digital "NeoBank" ini adalah sebuah simulasi arsitektur *backend* perbankan berbasis terminal yang dirancang untuk menerapkan konsep lanjutan Pemrograman Berorientasi Objek (OOP). Tujuan utama dari program ini adalah mengimplementasikan *Abstract Class*, *Interface*, *Encapsulation*, serta *Polymorphism (Method Overriding)* secara komprehensif. Selain itu, program ini juga berfokus pada manajemen siklus hidup objek melalui penerapan relasi antar kelas seperti *Composition*, *Aggregation*, dan *Association*.

Tantangan utama dalam pengembangan program ini adalah membangun hierarki perbankan yang aman (saldo tidak dapat diakses langsung), serta menyusun logika menu interaktif yang dapat mendemonstrasikan kelangsungan hidup suatu objek (misalnya nasabah, rekening, dan mutasi) ketika objek parent-nya dihapus atau ditiadakan dari memori sistem.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	<i>noRekening</i>	String	Menyimpan nomor identitas rekening milik nasabah (Protected).
2	saldo	double	Menyimpan nilai uang nasabah. Dientkapsulasi ketat (Private).
3	pin	String	Menyimpan kode rahasia untuk otorisasi login ke dalam rekening.
4	mutasi	BukuMutasi	Objek internal (<i>Composition</i>) untuk mencatat log transaksi rekening.
5	BIAYA_ADMIN	double	Menyimpan potongan tetap pada transaksi tarik tunai (RekeningReguler).
6	MIN_TARIK	double	Batas minimal penarikan khusus untuk jenis RekeningPrioritas.
7	daftarRekening	Array of Rekening	Menyimpan objek rekening independen ke profil nasabah (<i>Aggregation</i>).

III. Constructor dan Method

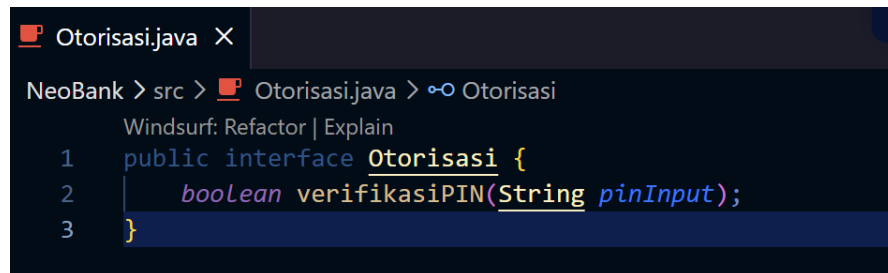
No	Nama Metode	Jenis Metode	Fungsi
1	Rekening(), Nasabah()	<i>Constructor</i>	Menginisialisasi atribut dasar serta membangun objek internal (BukuMutasi).
2	verifikasiPIN(String)	<i>Procedural / Implemented</i>	Memvalidasi input pengguna dengan PIN yang terdaftar di dalam sistem.
3	setor(double)	Procedural	Menambah saldo rekening dan mencatat aktivitas ke objek BukuMutasi.
4	tarik(double)	<i>Abstract / Overridden</i>	Mengurangi saldo dengan logika pemotongan yang berbeda pada tiap jenis akun.
5	tambahRekening() ()	<i>Procedural</i>	Memasukkan objek Rekening eksternal ke dalam array profil Nasabah.
6	laporKeluhan()	<i>Procedural</i>	Mendemonstrasikan <i>Association</i> dengan mengirim pesan ke objek CustomerService.

IV. Dokumentasi dan Pembahasan Code

4.1. Pembuatan Interface dan Abstract Class

Pada bagian ini, sistem mendefinisikan standar keamanan melalui antarmuka (Otorisasi.java) dan cetak biru enkapsulasi data perbankan (Rekening.java).

Kode Otorisasi.java:



```
Otorisasi.java X
NeoBank > src > Otorisasi.java > Otorisasi
Windsurf: Refactor | Explain
1 public interface Otorisasi {
2     boolean verifikasiPIN(String pinInput);
3 }
```

Kode Rekening.java:

```

Rekening.java X
NeoBank > src > Rekening.java > Rekening > getNoRekening()
Windsurf: Refactor | Explain
1 public abstract class Rekening implements Otorisasi {
2     protected String pin
3     protected String
4     private double Source: THT MODUL 9_e0f74984
5     private String pin;
6     private BukuMutasi mutasi;
7
8     public Rekening(String noRekening, String namaPemilik, double saldoAwal, String pin) {
9         this.noRekening = noRekening;
10        this.namaPemilik = namaPemilik;
11        this.saldo = saldoAwal;
12        this.pin = pin;
13        this.mutasi = new BukuMutasi();
14    }
15
16    Windsurf: Refactor | Explain | Generate Javadoc | X
17    public void setor(double jumlah) {
18        if (jumlah > 0) {
19            saldo += jumlah;
20            // Format angka dengan pemisah titik
21            String info = String.format(new java.util.Locale(language: "id", country: "ID"),
22                format: "Setor tunai: %,0f | Saldo akhir: %,0f", jumlah, saldo);
23            mutasi.catat(info);
24        }
25
26    public abstract void tarik(double jumlah);
27
28    Windsurf: Refactor | Explain | Generate Javadoc | X
29    protected void kurangiSaldo(double jumlah) {
30        this.saldo -= jumlah;
31    }
32
33    Windsurf: Refactor | Explain | Generate Javadoc | X
34    public double getSaldo() {
35        return saldo;
36    }
37
38    Windsurf: Refactor | Explain | Generate Javadoc | X
39    public BukuMutasi getMutasi() {
40        return mutasi;
41    }
42
43    Windsurf: Refactor | Explain | Generate Javadoc | X
44    @Override
45    public boolean verifikasiPIN(String pinInput) {
46        return this.pin.equals(pinInput);
47    }
48
49    Windsurf: Refactor | Explain | Generate Javadoc | X
50    public String getNoRekening() {
51        return noRekening;
52    }
53
54 }

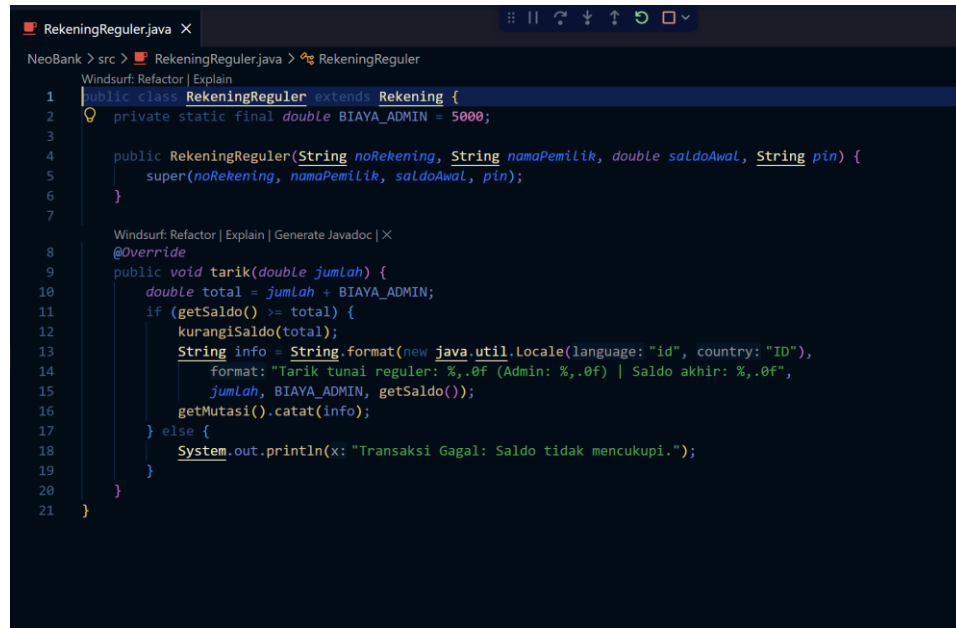
```

Pembahasan: Interface Otorisasi mewajibkan kelas turunannya mengimplementasikan fungsi keamanan PIN. Kelas Rekening dibangun sebagai abstrak karena secara logis tidak ada rekening tanpa spesifikasi jenis. Enkapsulasi ditegakkan dengan ketat: variabel saldo bersifat *private*, sehingga hanya dapat dimanipulasi melalui metode khusus (setor dan tarik). Selain itu, di dalam *constructor*, terdapat instansiasi BukuMutasi, yang merupakan wujud nyata dari konsep **Composition** (objek mutasi hidup dan mati bersama objek rekening pembentuknya).

4.2. Pewarisan dan Polimorfisme (Jenis Rekening)

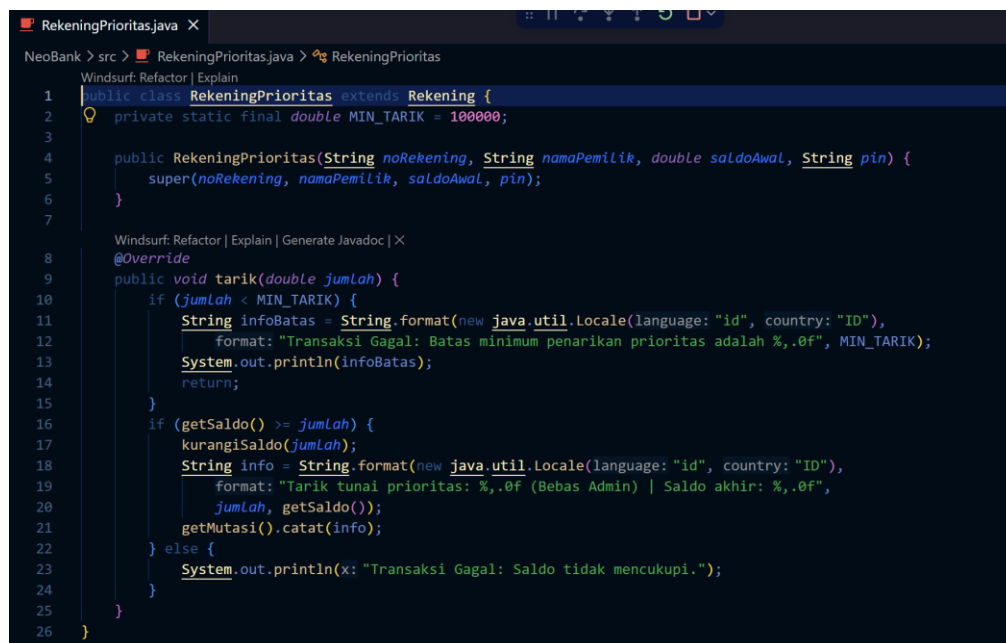
Sistem memiliki dua jenis rekening turunan yang mengimplementasikan polimorfisme pada *method* tarik().

Kode RekeningReguler.java:



```
RekeningReguler.java X
NeoBank > src > RekeningReguler.java > RekeningReguler
Windsurf: Refactor | Explain
1 public class RekeningReguler extends Rekening {
2     private static final double BIAYA_ADMIN = 5000;
3
4     public RekeningReguler(String noRekening, String namaPemilik, double saldoAwal, String pin) {
5         super(noRekening, namaPemilik, saldoAwal, pin);
6     }
7
8     Windsurf: Refactor | Explain | Generate Javadoc | X
9     @Override
10    public void tarik(double jumlah) {
11        double total = jumlah + BIAYA_ADMIN;
12        if (getSaldo() >= total) {
13            kurangiSaldo(total);
14            String info = String.format(new java.util.Locale(language: "id", country: "ID"),
15                format: "Tarik tunai reguler: %,0f (Admin: %,0f) | Saldo akhir: %,0f",
16                jumlah, BIAYA_ADMIN, getSaldo());
17            getMutasi().catat(info);
18        } else {
19            System.out.println(x: "Transaksi Gagal: Saldo tidak mencukupi.");
20        }
21    }
22 }
```

Kode RekeningPrioritas.java:



```
RekeningPrioritas.java X
NeoBank > src > RekeningPrioritas.java > RekeningPrioritas
Windsurf: Refactor | Explain
1 public class RekeningPrioritas extends Rekening {
2     private static final double MIN_TARIK = 100000;
3
4     public RekeningPrioritas(String noRekening, String namaPemilik, double saldoAwal, String pin) {
5         super(noRekening, namaPemilik, saldoAwal, pin);
6     }
7
8     Windsurf: Refactor | Explain | Generate Javadoc | X
9     @Override
10    public void tarik(double jumlah) {
11        if (jumlah < MIN_TARIK) {
12            String infoBatas = String.format(new java.util.Locale(language: "id", country: "ID"),
13                format: "Transaksi Gagal: Batas minimum penarikan prioritas adalah %,0f", MIN_TARIK);
14            System.out.println(infoBatas);
15            return;
16        }
17        if (getSaldo() >= jumlah) {
18            kurangiSaldo(jumlah);
19            String info = String.format(new java.util.Locale(language: "id", country: "ID"),
20                format: "Tarik tunai prioritas: %,0f (Bebas Admin) | Saldo akhir: %,0f",
21                jumlah, getSaldo());
22            getMutasi().catat(info);
23        } else {
24            System.out.println(x: "Transaksi Gagal: Saldo tidak mencukupi.");
25        }
26    }
27 }
```

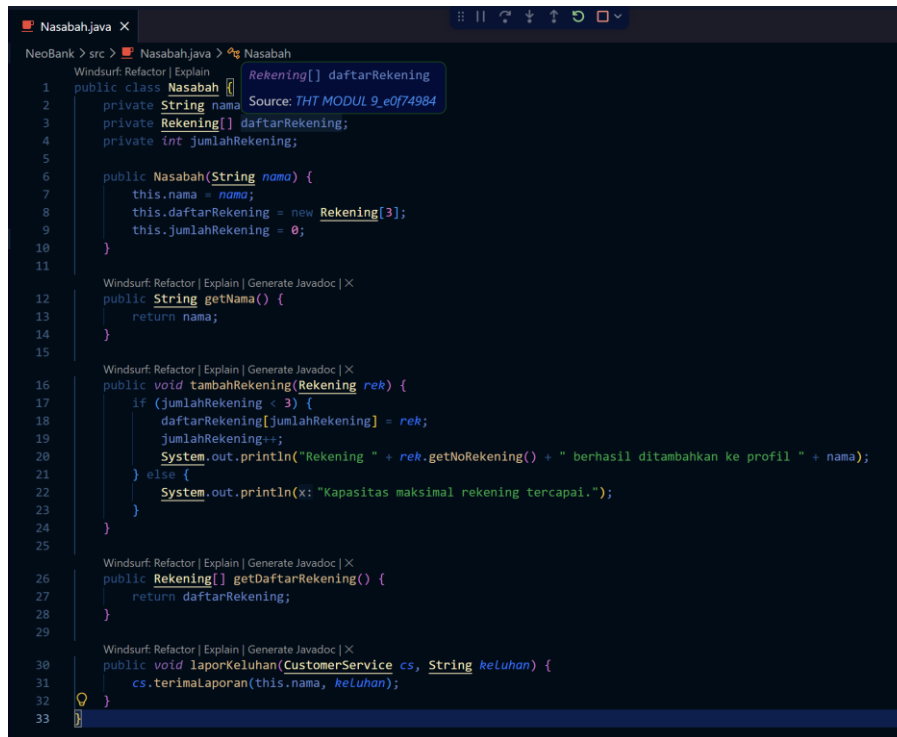
Pembahasan:

RekeningReguler dan RekeningPrioritas adalah *concrete class* yang mewarisi sifat abstrak dari Rekening. Polimorfisme (*Method Overriding*) terlihat jelas pada metode tarik(). Kelas reguler akan selalu memotong saldo ekstra untuk biaya admin, sedangkan

kelas prioritas bebas biaya admin namun menyertakan validasi ketat batas minimum penarikan yang telah ditetapkan (MIN_TARIK).

4.3. Implementasi Agregasi dan Asosiasi

Kode Nasabah.java:



```
1 public class Nasabah {
2     private String nama;
3     private Rekening[] daftarRekening;
4     private int jumlahRekening;
5
6     public Nasabah(String nama) {
7         this.nama = nama;
8         this.daftarRekening = new Rekening[3];
9         this.jumlahRekening = 0;
10    }
11
12    public String getName() {
13        return nama;
14    }
15
16    public void tambahRekening(Rekening rek) {
17        if (jumlahRekening < 3) {
18            daftarRekening[jumlahRekening] = rek;
19            jumlahRekening++;
20            System.out.println("Rekening " + rek.getNoRekening() + " berhasil ditambahkan ke profil " + nama);
21        } else {
22            System.out.println("Kapasitas maksimal rekening tercapai.");
23        }
24    }
25
26    public Rekening[] getDaftarRekening() {
27        return daftarRekening;
28    }
29
30    public void laporKeluhan(CustomerService cs, String keluhan) {
31        cs.terimaLaporan(this.nama, keluhan);
32    }
33 }
```

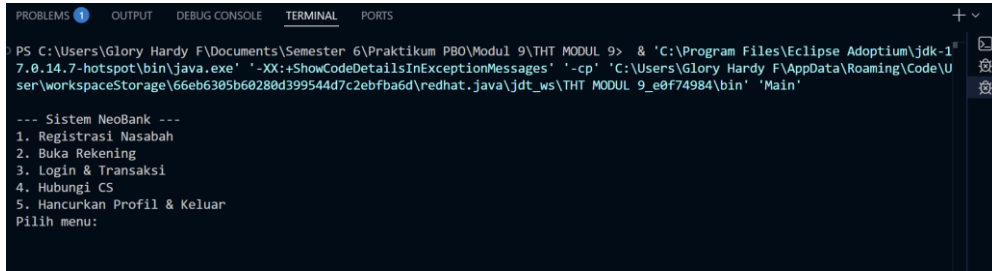
Pembahasan:

Hubungan Aggregation terlihat pada array `daftarRekening` di kelas `Nasabah`. `Rekening` disisipkan melalui metode *setter/adder*, menandakan status *loose-coupling*; nasabah memiliki rekening, namun eksistensi rekening tersebut tidak bergantung sepenuhnya pada eksistensi objek nasabah. Sementara itu, Association ditunjukkan melalui metode `laporKeluhan()`, di mana `Nasabah` dan `CustomerService` berinteraksi dan berdiri sendiri tanpa ada konsep kepemilikan mutlak.

4.4. Bukti Siklus Hidup Objek (Main Class)

Berdasarkan eksperimen di dalam eksekusi *looping* `Main.java`, manajemen memori pada relasi PBO berjalan sesuai landasan teori:

- Saat objek nasabah di-set menjadi null (opsi Hancurkan Profil), profil `Nasabah` musnah dari memori. Namun, karena relasi `Nasabah` dengan `Rekening` adalah *Aggregation*, objek `Rekening` yang telah dideklarasikan secara independen (melalui variabel `rekeningSimpanan`) tetap hidup di memori.
- Sebaliknya, karena objek `BukuMutasi` diinstansiasi langsung di dalam *constructor* (relasi *Composition*), jika objek `Rekening` tersebut dihancurkan, maka entitas `BukuMutasi` di dalamnya dipastikan akan ikut musnah tanpa sisa karena tidak memiliki eksistensi mandiri di luar kelas parent-nya.



```
PS C:\Users\Glory Hardy F\Documents\Semester 6\Praktikum PBO\Modul 9\THT MODUL 9> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.14.7-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Glory Hardy F\AppData\Roaming\Code\U
ser\workspaceStorage\66eb6305b60280d399544d7c2ebfba6d\redhat.java\jdt_ws\THT MODUL 9_e0f74984\bin' 'Main'

--- Sistem NeoBank ---
1. Registrasi Nasabah
2. Buka Rekening
3. Login & Transaksi
4. Hubungi CS
5. Hancurkan Profil & Keluar
Pilih menu:
```

V. Kesimpulan

Berdasarkan hasil perancangan arsitektur perbankan NeoBank, dapat ditarik kesimpulan sebagai berikut:

1. Abstraksi dan Hierarki: Penerapan *Abstract Class* sangat vital sebagai cetak biru pusat untuk memaksa standar struktural pada berbagai tipe rekening (Reguler dan Prioritas) tanpa mengizinkan instansiasi langsung pada objek generik.
2. Polimorfisme Fungsional: *Overriding* metode pada kelas turunan memungkinkan respon fungsional yang berbeda (pengenaan admin dan verifikasi batas nominal) dengan pemanggilan nama metode yang sama.
3. Manajemen Siklus Objek (Lifecycle): Eksperimen membuktikan bahwa desain relasi mempengaruhi nasib data di memori. Data BukuMutasi terikat mati (*Composition*) dan akan ikut terhapus bersama entitas Rekening. Sebaliknya, entitas Rekening yang dikelola secara *Aggregation* dapat dibiarkan bertahan pada memori meskipun entitas Nasabah dihapus.

VI. Daftar Pustaka

- Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education. (Referensi untuk pemahaman tipe data dan operator aritmatika).
- Oracle. (2026). *Java SE Documentation: Class Scanner*. [Online]. Tersedia di: <https://docs.oracle.com/en/java/> (Referensi untuk penggunaan input stream).
- Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.